

Rapport d'analyse du code - PID Table Robovaly

Date du rapport: 20 aout 2026. Analyse statique realisee a partir des fichiers src/main.cpp, src/tof.*, src/ball_position.*, src/controller.*, src/servo_cmd.* et src/interface_web.*. Aucune modification du code source n'a ete effectuee.

Synthese

Le projet est un firmware PlatformIO pour ESP32/Arduino. Il pilote une table a balle avec deux capteurs Time-of-Flight VL53L0X, un servo PWM et un controleur PID. L'ESP32 expose aussi un point d'accès WiFi et une interface web pour la calibration, la visualisation en temps reel, le reglage PID et les parametres avances.

Architecture generale

Module	Role
main.cpp	Initialise le serial, les capteurs TOF, les reglages persistants, le controleur et l'interface web. Cree une tache FreeRTOS pour appeler update_controller().
tof.cpp/h	Configure les deux VL53L0X sur bus I2C, leur assigne des adresses distinctes et produit des mesures horodatees dans une tache dediee.
ball_position.cpp/h	Valide les mesures, applique la calibration TOF, fusionne les deux estimations de position et filtre position/vitesse avec un estimateur alpha-beta.
controller.cpp/h	Calcule la commande PID, applique deadbands, limite de vitesse, detection de stabilite, mode idle et retour progressif en cas de balle perdue.
servo_cmd.cpp/h	Convertit les angles en impulsions PWM 50 Hz, applique limites mecaniques/theoriques et gere l'offset de neutre.
interface_web.cpp/h	Heberge l'interface utilisateur, les routes JSON, les flux de calibration et les Preferences NVS.

Cycle d'execution et acquisition

- Au demarrage, `init_tof(false)` lance la tache d'acquisition des TOF;
`load_startup_persistent_settings()` restaure calibration et reglages; `init_controller()` initialise le servo; `launch_interface_web()` demarre la tache web.
- La tache controleur, epinglee sur le coeur 1, appelle `update_controller()` avec la periode configurable `get_controller_period_ms()`, par default 30 ms.
- La boucle Arduino principale dort; le fonctionnement utile est porte par les taches FreeRTOS TOF, controleur et web.
- La tache web, sur le coeur 0, sert les pages HTML et les endpoints JSON, puis ajuste automatiquement le mode de controle selon la presence d'un client WiFi.

Acquisition TOF

- `tof.cpp` coupe d'abord les deux capteurs via XSHUT, demarre chaque VL53L0X a l'adresse par default 0x29, puis les deplace vers 0x30 et 0x31.
- Les mesures sont lancees en alternance: START/WAIT/READ TOF 1 puis START/WAIT/READ TOF 2. Le budget de mesure est 20 ms, avec un slot de 25 ms et un polling a 1 ms.
- Les valeurs, etats d'initialisation et timestamps sont proteges par `portMUX_TYPE`, ce qui evite les lectures partiellement incoherentes entre taches.

Calcul de position

- `update_tof_distances()` ne traite une mesure que si son timestamp a change, invalide un capteur muet apres 120 ms et expose -1 si la mesure n'est plus fiable.
- `linearise_tof_measure()` transforme les mesures brutes grace a une calibration piecewise lineaire: distance au bord du FOV, puis points 145, 72 et 0 mm si disponibles.
- `compute_ball_position()` convertit TOF1 en position `table_length - distance`, TOF2 en distance directe, puis fusionne les deux positions par moyenne ponderee.
- Le poids combine une confiance liee a la proximite du bord de FOV et un poids lie au bruit mesure pour chaque capteur et chaque zone de la table.
- Si les deux capteurs donnent simultanement une distance inferieure a 80 mm, le code considere la situation incoherente et declare la balle perdue.

Estimation de vitesse

- `compute_ball_speed()` utilise `latest_tof_input_update_ms` comme temps d'echantillonnage, pas simplement `millis()`, ce qui aligne la vitesse avec les nouvelles mesures.
- Le filtre alpha-beta predit position et vitesse, puis corrige avec l'innovation mesuree. Alpha et beta sont reduits quand le bruit local augmente.
- Un grand ecart entre prediction et mesure reduit encore la confiance, ce qui amortit les sauts brusques. Apres 250 ms sans echantillon coherent, l'estimateur est reinitialise.

Contrôle PID, servo et interface web

Contrôleur PID

- Le PID contrôle l'angle du servo à partir de l'erreur référence - position et de la vitesse estimée: $\text{commande} = k_p \cdot \text{erreur} + k_i \cdot \text{integrale} - k_d \cdot \text{vitesse}$.
- Les erreurs proches de zéro sont atténuées par `soften_deadband_signal()`, puis l'intégrale est bornée à $\pm 3000 \text{ mm.s}$ pour limiter le windup.
- La commande est convertie en angle autour du neutre du servo, puis bornée entre les limites mécaniques configurées.
- Le système détecte la stabilité lorsque la position et la vitesse restent dans leurs deadbands pendant `stable_time_ms`. Il passe alors en idle et commande le neutre tant que l'hystérésis de sortie n'est pas dépassée.
- En balle perdue, le contrôleur invalide la mise à jour, remet certains termes à zéro après `lost_iter` cycles, puis revient lentement au neutre après `lost_ball_delay_ms`.

Servo

- `servo_cmd.cpp` produit un PWM 50 Hz en résolution 16 bits sur la broche 18 par défaut.
- Le code distingue plage théorique, limites opérationnelles et offset de neutre en microsecondes. C'est utile pour s'adapter au montage réel sans changer l'algorithme PID.
- La fonction `set_servo_calibration_pulse_us()` permet de commander directement une impulsion pendant la calibration du neutre.

Interface web et persistance

- L'ESP32 crée le point d'accès `Kit_Robovaly` avec le mot de passe `robovaly123`, puis sert l'interface sur le port 80.
- Les pages embarquées permettent: accueil/calibration initiale, calibration servo, calibration TOF, vérification, mesure du bruit, page principale temps réel et page avancée.
- Les Préférences NVS sont séparées en namespaces: `tof_cal` pour calibration, `ctrl` pour référence et gains PID, `advanced` pour paramètres avancés. Des versions de schéma filtrent les anciennes données.
- Quand aucun client WiFi n'est connecté et que les capteurs sont calibrés, le firmware recharge les réglages sauvegardés et active automatiquement la stabilisation.

Points forts

- Separation fonctionnelle nette: acquisition, estimation, controle, servo et interface web sont dans des modules distincts avec des headers simples.
- Architecture temps reel adaptee a l'ESP32: acquisition TOF, controle et serveur web tournent dans des taches separees, avec affinites de coeur explicites.
- Gestion robuste de la calibration: workflow guide, sauvegarde en NVS, versionnement de schema et possibilite de recalibrer separement servo, TOF ou bruit.
- Fusion capteur intelligente: le code ne se contente pas d'une moyenne brute; il pondere selon le FOV et selon le bruit local mesure.
- Filtrage dynamique: les parametres alpha-beta s'adaptent au bruit local et reduisent la confiance lors des innovations excessives.
- Protection contre plusieurs cas limites: timeouts capteurs, mesures invalides, balle perdue, retour progressif au neutre, saturation vitesse, saturation servo et limite d'integrale.
- Interface pedagogique riche: donnees temps reel, courbes, reglages PID, commandes manuelles et calibration sont accessibles sans outil externe.
- Les setters valident largement les plages d'entree, ce qui reduit le risque de configurer des parametres absurdes depuis l'interface web.

Points fragiles ou a surveiller

- Plusieurs variables globales partagees entre taches ne sont pas protegees hors module TOF. Les etats du controleur, de position et de calibration peuvent etre lus/ecrits par la tache web et la tache controleur sans verrou explicite.
- interface_web.cpp est tres volumineux et melange HTML/CSS/JS, logique HTTP, calibration, persistance et controle de modes. La maintenance deviendra difficile si de nouvelles fonctions sont ajoutees.
- Les endpoints utilisent HTTP GET pour modifier l'etat et sauvegarder des reglages. C'est simple pour un atelier, mais fragile si plusieurs clients ou rafraichissements accidentels interagissent avec le systeme.
- Le point d'accès utilise un SSID et un mot de passe constants dans le firmware. C'est acceptable pour un kit local, moins pour un usage partage ou public.
- Les chaines JSON sont construites manuellement. Comme les erreurs/instructions ne sont pas echappees par une librairie JSON, une chaine contenant guillemets ou caracteres speciaux pourrait casser la reponse.
- Certains anciens parametres dans ball_position.cpp semblent inutilises ou herites (BALL_RADIUS_MM, TABLE_INCERT_MM, MAX/MIN_VALUE_TOF*, offsets). Cela peut brouiller la lecture et creer de fausses pistes.
- Le filtre servo est configure avec SERVO_FILTER_ALPHA = 1.0, donc le lissage repose surtout sur max_step_deg; le nom smooth_servo_angle peut laisser croire a un filtrage plus progressif.
- La calibration suppose une relation monotone des mesures TOF et des points fixes 0/72/145/218/290 mm. Si la geometrie ou la table change fortement, plusieurs constantes et hypotheses doivent etre revisees.
- La generation des pages HTML en PROGMEM rend le firmware autonome, mais complique les tests automatiques et la revue fine du JavaScript.
- Il n'y a pas de tests unitaires visibles pour les fonctions critiques de linearisation, fusion, filtre alpha-beta, PID et validation des parametres.

Conclusion et recommandations

Conclusion

Le code est globalement coherent pour un prototype pedagogique avance: il combine une vraie logique de calibration, une estimation de position/vitesse plus robuste qu'une lecture directe, un controle PID raisonnablement protege et une interface web complete. Le systeme est deja bien pense pour l'atelier: il est autonome, configurable et observable.

Les fragilites principales ne sont pas des erreurs evidentes de logique, mais des risques d'evolution: concurrence entre taches, taille de l'interface web, JSON manuel, absence de tests et dependance forte aux hypotheses mecaniques/calibration.

Priorites conseillees sans changer le comportement

- Documenter explicitement le repere de position: TOF1 mesure depuis une extremite et TOF2 depuis l'autre, avec position finale en mm sur la table.
- Ajouter des tests hors materiel pour `linearise_measure()`, `set_tof_calibration()`, `compute_ball_position()` avec valeurs injectees, et la logique PID sur scenarios simples.
- Introduire progressivement un petit module de synchronisation ou snapshots atomiques pour les donnees partagees entre web et controleur.
- Extraire les pages web dans des fichiers sources separes ou dans un pipeline minimal afin de faciliter la revue du front sans toucher au firmware logique.
- Remplacer a terme la construction manuelle de JSON par une petite fonction d'echappement ou une librairie adaptee a l'ESP32.

Fichiers inspectes

platformio.ini; src/main.cpp; src/tof.cpp/h; src/ball_position.cpp/h; src/controller.cpp/h; src/servo_cmd.cpp/h; src/interface_web.cpp/h. Les dossiers include, lib et test ne contiennent ici que des README.